

ESP-DL vs ESP-TFLite-Micro: Neural Networks Inference on ESP32 Devices

Simone Reale

Alma Mater Studiorum University of Bologna

Bologna, Italy

simone.reale3@studio.unibo.it

Abstract—As the deployment of Edge Artificial Intelligence (Edge AI) on resource-constrained IoT devices accelerates, selecting the optimal neural network inference engine is critical for system efficiency. This technical report presents a comprehensive comparative analysis of two prominent inference libraries tailored for the ESP32 microcontroller: Espressif’s native ESP-DL and Google’s TensorFlow Lite for Microcontrollers (ESP-TFLite-micro). The study evaluates these frameworks across two primary dimensions. Qualitatively, we examine the supported functionalities, layer operations, framework flexibility, and ease of integration within the ESP-IDF ecosystem. Quantitatively, we benchmark their execution performance, focusing heavily on memory footprint (model size and RAM utilization) and inference latency. By juxtaposing the specialized, hardware-optimized nature of ESP-DL against the broad, ecosystem-rich versatility of ESP-TFLite-micro, this report provides developers with actionable, data-driven guidelines for selecting the most appropriate framework based on specific application requirements and strict hardware constraints.

I. OVERVIEW

The migration of artificial intelligence workloads from centralized cloud infrastructures to the network edge represents a critical evolution in distributed system architecture. In the context of the Internet of Things (IoT), this development is motivated by the necessity for immediate responsiveness, improved data privacy, and the reduction of bandwidth constraints. The ESP32 series of microcontrollers by Espressif Systems serves as a foundational hardware platform for this transition, providing a balance of multi-core processing capabilities, integrated wireless communication, and low power consumption. However, deploying neural networks on these devices presents substantial engineering challenges. Standard microcontrollers typically feature severely constrained resources, such as the 520 KB of on-chip SRAM found on the different ESP32 variants. Consequently, selecting an appropriate inference engine becomes a critical architectural decision that determines the feasibility and efficiency of the embedded system. Two primary frameworks define the current ESP32 inference ecosystem: Espressif’s native **ESP-DL** and Google’s **TensorFlow Lite for Microcontrollers (ESP-TFLite-micro)**. These libraries embody divergent architectural methodologies: a statically compiled, hardware-specific approach versus a dynamically interpreted, platform-agnostic model. In this report, they are analysed in their supported features (Sections I-A, I-B), and compared in terms of memory footprint and

inference time on the *ESP32-WROOM-32* and the *ESP32-S3 N16R8* microcontrollers (Section II).



A. ESP-TFLite-Micro

TensorFlow Lite for Microcontrollers (TFLM), integrated into the Espressif ecosystem as the **ESP-TFLite-micro** component, provides a highly adaptable framework designed for hardware with limited memory capacity. The library operates independently of an underlying operating system or standard dynamic memory allocators, ensuring high portability across various 32-bit architectures. In contrast to the static allocation strategy of ESP-DL, TFLM utilizes an interpreter-based architecture. Models are serialized into a standard *.tflite* format, converted into a C byte array, and typically stored in the read-only program memory of the device. During runtime execution, the interpreter requires a pre-allocated *Tensor Arena*. This arena is a contiguous block of RAM utilized to store input, output, and intermediate network tensors. The precise dimension of this memory block depends on the application and an inaccurate sizing can result in either wasted memory resources or runtime initialization failures.

The core advantage of TFLM is its standardized development workflow and extensive ecosystem integration. It establishes a continuous, end-to-end pipeline from model design and training in a standard TensorFlow or Keras environment down to embedded deployment via a *.tflite* FlatBuffer file. This workflow natively supports mature optimization techniques such as *Post-Training Quantization (PTQ)* and *Quantization-Aware Training (QAT)*, which are critical for compressing models into the tight SRAM and flash constraints of edge devices.

To address the performance limitations inherent to a platform-agnostic interpreter, Espressif incorporates the **ESP-NN** library directly into the ESP-IDF port of TFLM. Instead

of relying on TFLM’s default reference kernels—which are written in pure C++ and designed strictly for cross-platform portability rather than speed—ESP-NN acts as a highly tuned, hardware-aware computational backend. It supplies optimized C and assembly-level implementations for the most computationally expensive neural network operators, including 2D convolutions, depthwise separable convolutions, fully connected (dense) layers, pooling functions, and activations like Softmax.

When the TFLM interpreter builds the execution graph at initialization, its operator resolver (*MicroOpResolver*) intercepts standard operation calls and redirects them to these optimized ESP-NN equivalents. The true power of ESP-NN lies in its ability to exploit target-specific silicon microarchitectures. For example, when deployed on the ESP32-S3, ESP-NN extensively utilizes the Xtensa LX7 core’s custom AI Vector Instructions (*SIMD* - *Single Instruction, Multiple Data*). This hardware acceleration enables the processor to execute multiple 8-bit or 16-bit Multiply-Accumulate (MAC) operations in a single clock cycle, dramatically reducing the inference latency of `int8` quantized models. Additionally, ESP-NN kernels are explicitly written to optimize the ESP32’s memory hierarchy, utilizing loop unrolling and efficient data alignment to minimize cache misses and maximize data throughput from SRAM to the CPU registers.



B. ESP-DL

ESP-DL is a specialized library engineered to maximize the hardware capabilities of the ESP series, particularly AI-accelerated chipsets like the *ESP32-S3* and *ESP32-P4*, with support also for the standard *ESP32* chipsets. The architecture of the framework is optimized to minimize execution overhead, prioritizing low inference latency and strict memory efficiency over cross-platform portability. A primary feature of ESP-DL is its proprietary model format, denoted by the `.espdl` extension. This format relies on *FlatBuffers* instead of usual *Protobuf* to make it more lightweight and facilitate zero-copy deserialization. Protobuf requires an explicit decoding step where the data structure must be completely deserialized and copied into newly allocated memory objects before the application can use it. With *FlatBuffers*, by enabling the CPU to access model weights directly from flash memory or a RAM buffer without requiring an intermediate parsing phase, ESP-DL substantially reduces initialization time and runtime memory consumption.

Furthermore, resource management in ESP-DL depends on a static memory planner. This planner evaluates the architecture of the neural network to optimally allocate memory buffers prior to the initial inference. This mechanism avoids dynamic memory allocation, thereby preventing heap fragmentation. It also optimizes the placement of layer buffers between

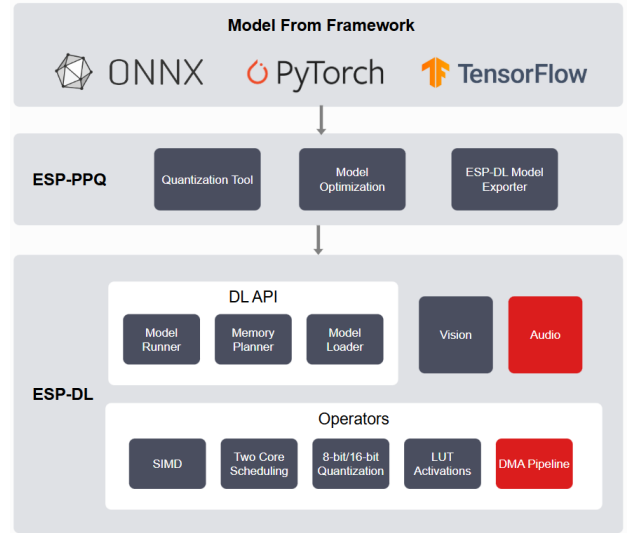


Fig. 1. ESP-DL Architecture

fast internal SRAM and external storage options like PSRAM or Flash. Internally, the framework combines high-level C++ APIs with hand-optimized assembly code for computational kernels. To mitigate performance bottlenecks, ESP-DL computes complex non-linear activation functions via an 8-bit *Look-Up Table (LUT)* methodology, which guarantees constant computational complexity. Model quantization is executed through the *ESP Post-training Production Quantization (ESP-PPQ)* tool, an optimized and open-source tool that supports both 8-bit and 16-bit precision representations. Additionally, ESP-DL incorporates native dual-core scheduling for intensive operations, automatically distributing workloads across both CPU cores to decrease inference latency.

II. PERFORMANCE EVALUATION

To rigorously assess the viability of ESP-DL and ESP-TFLite-Micro, a quantitative analysis is required alongside the qualitative architectural review, focusing on the two most critical constraints in embedded machine learning: memory utilization and inference latency.

To evaluate the computational efficiency and inference accuracy of the proposed frameworks, a regression task was implemented focusing on the approximation of a periodic signal. The baseline model consists of a *Multi-Layer Perceptron (MLP)* architecture structured with an input layer, three sequential hidden layers containing 32, 64, and 128 neurons respectively, and a single output neuron. The network was trained to reconstruct the sine function $f(x) = \sin(x)$ within the domain $x \in [0, 2\pi]$ (Figures 2, 3).

Post-training, the model was subjected to a rigorous quantization process to meet the constraints of the ESP32 hardware environment. Specifically, two distinct optimization pipelines were employed: the standard *TensorFlow Lite quantization suite* for integration with the ESP-TFLite-micro library, and the *ESP-PPQ* tool optimized for the ESP-DL library.

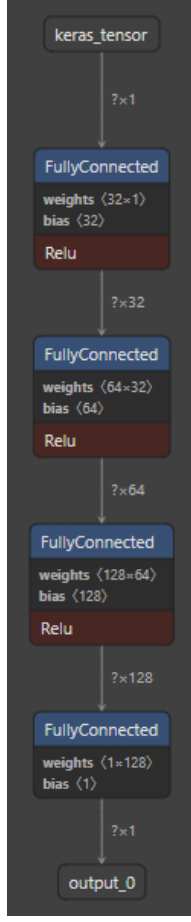


Fig. 2. Sine wave predictor MLP model visualization in Netron.

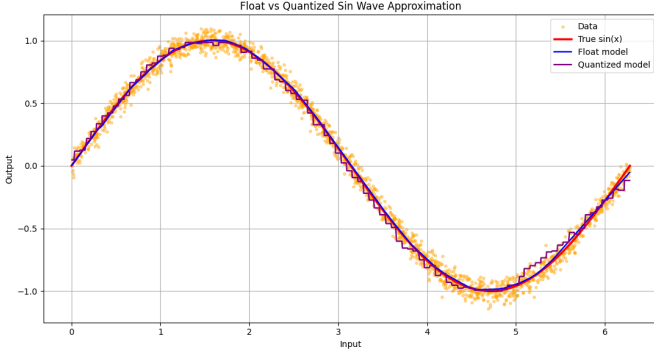


Fig. 3. Sine wave predictor performance evaluation ($\sin(x)$ vs float32 model vs int8 model).

A. Memory Footprint

Deploying neural networks on resource-constrained Edge IoT devices, such as the ESP32 microcontroller family, requires a rigorous evaluation of memory allocation strategies, specifically delineating between non-volatile memory (Flash) and volatile memory (SRAM). Both ESP-TFLite-micro and ESP-DL leverage 8-bit quantization (int8) to compress model weights and utilize FlatBuffers for zero-copy

deserialization directly from Flash memory. However, their empirical memory utilization diverges significantly due to their distinct FlatBuffer schemas and runtime execution paradigms.

In the ESP-TFLite-micro ecosystem, the model is serialized using Google’s standardized, platform-agnostic FlatBuffer schema. While this ensures board compatibility, it introduces substantial spatial overhead. To execute the `.tflite` file, the application must compile the TensorFlow Lite Micro interpreter, the memory planner, and specific *MicroMutableOpResolvers* instances into Flash memory. More critically, SRAM utilization is dictated by the *Tensor Arena*, a contiguous block of statically allocated volatile memory required by the interpreter to manage the generalized computational graph at runtime. This arena must be sized to accommodate input/output tensors, intermediate activation buffers, internal state objects, and quantization parameters. Consequently, the Tensor Arena demands a disproportionate SRAM allocation to prevent runtime segmentation faults, leading to a memory profile that is notably inflated.

Conversely, the ESP-DL library employs a proprietary, highly optimized FlatBuffer schema explicitly tailored for Espressif silicon. Utilizing the ESP deployment toolchain, the neural network is compiled into this custom schema, which maps directly to the underlying hardware-accelerated kernels (such as the ESP-NN library). Because the `.espdl` format is strictly coupled with the target architecture, it entirely circumvents the need for a bloated, generalized interpreter engine in Flash. Furthermore, ESP-DL bypasses the monolithic Tensor Arena approach. The runtime extracts the execution graph and memory requirements directly from the optimized FlatBuffer, executing operators via bare-metal function calls and reusing intermediate SRAM buffers with high determinism. By eliminating the interpreter abstraction layer while retaining the serialization benefits of FlatBuffers, ESP-DL achieves a volatile memory footprint that closely approaches the theoretical lower limit of the network’s structural size, thereby preserving critical SRAM for concurrent IoT tasks.

Empirical measurements reflect the description above, showing a reduction in non-volatile Flash memory in favour of ESP-DL, which is directly attributable to the architectural differences in their respective serialization schemas.

<i>ESP-DL</i>	<i>ESP-TFLite-micro</i>
15.34 kB	18.84 kB

Because both frameworks leverage identical 8-bit integer (int8) quantization, the baseline payload of neural weights and biases remains constant. The divergence arises from the structural overhead inherent to Google’s generalized FlatBuffer schema, which must embed cross-platform metadata and dynamic operator definitions. Conversely, the tightly coupled `.espdl` format strips away this generalized metadata, minimizing the schema size to exclusively map what is required for Espressif’s hardware kernels.

However, it is imperative to emphasize that while the static Flash footprints exhibit only a marginal difference, the dynamic SRAM allocation during execution – specifically the elimination of the interpreter’s Tensor Arena – remains the paramount advantage of the ESP-DL framework.

B. Inference latency

In the context of edge AI deployments on the ESP32 micro-controller architecture, inference latency constitutes a critical performance metric. A comparative analysis of execution times reveals underlying performance disparities between the ESP-TFLite-micro and ESP-DL frameworks, driven primarily by their execution paradigms. ESP-TFLite-micro natively operates on an interpreter-based model, which introduces inherent structural overhead during inference. For each forward pass, the processor must dynamically parse the serialized computational graph, resolve operations at runtime, and invoke generalized computational kernels, thereby consuming additional clock cycles.

On the other side, the ESP-DL library systematically minimizes structural latency by employing a bare-metal execution strategy. By utilizing a tightly coupled deployment toolchain, the neural network is mapped directly to the ESP-NN library, entirely bypassing the interpreter abstraction layer. Furthermore, ESP-DL is specifically engineered to leverage the custom hardware acceleration of Espressif silicon, such as the vector instructions and SIMD (Single Instruction, Multiple Data) extensions present in the ESP32-S3 SoC. This direct invocation of highly optimized, hardware-specific assembly routines allows ESP-DL to execute matrix multiplications and dense layer operations with substantially reduced computational delay compared to the unoptimized TFLite-micro baseline.

Target	<i>ESP-DL</i>	<i>TFLite-micro (with ESP-NN)</i>	<i>TFLite-micro (no ESP-NN)</i>
ESP32	–	0.802 <i>ms</i>	0.802 <i>ms</i>
ESP32-S3	0.488 <i>ms</i>	0.315 <i>ms</i>	0.541 <i>ms</i>

The empirical results presented in the table above provide crucial nuances to the theoretical performance models. Notably, ESP-DL benchmarking is absent for the standard ESP32 target. This omission is dictated by strict hardware constraints, since the standard ESP32 configuration utilized for testing lacks Pseudo-Static RAM (PSRAM). Additionally, regarding the standard ESP32 target, it is imperative to specify that the recorded latency of 0.802 ms for ESP-TFLite-micro remains the same also when enabling ESP-NN optimizations, given the lack of ESP-S3 SIMD accelerator instructions. The absolute inference time is subject to measurable micro-variations depending heavily on the cache memory alignment of the Tensor Arena and the specified opset of the resolver. Because the arena serves as a monolithic contiguous block for all intermediate activations and internal states, its specific alignment boundary relative to the CPU cache lines

directly dictates memory fetch efficiency. Optimal alignment significantly mitigates cache misses during matrix operations, whereas suboptimal alignment can introduce variable latency penalties during the forward pass.

Furthermore, the data recorded for the ESP32-S3 introduces a multi-faceted performance hierarchy. As anticipated by the architectural analysis, the bare-metal *ESP-DL* framework at 0.488 ms effectively outperforms the unoptimized *TFLite-micro (no ESP-NN)* implementation at 0.541 ms. This performance gap successfully isolates and quantifies the inherent latency penalty introduced by the runtime interpreter and the reliance on generic C++ reference kernels when hardware acceleration is disabled.

However, a counter-intuitive observation emerges when hardware acceleration is selectively enabled: *TFLite-micro (with ESP-NN)* at 0.315 ms demonstrably outperforms the bare-metal *ESP-DL* framework. While ESP-DL lacks an interpreter overhead, this benchmark highlights the profound impact of software-level graph optimization. The heavily modified Espressif port of TFLite-micro bypasses standard C++ implementations for supported layers and directly invokes with *ESP-NN* the exact same hardware-accelerated *SIMD* kernels used by ESP-DL. The latency advantage observed for TFLite-micro is thus attributed to the maturity of the TensorFlow conversion pipeline, which performs aggressive operator fusion and leverages the contiguous memory allocation of the Tensor Arena to maximize CPU cache hits. Consequently, while ESP-DL remains fundamentally leaner in volatile memory footprint, TFLite-micro – when bridged directly to ESP-NN – capitalizes on mature graph optimizations to achieve superior inference speeds on supported hardware.

CONCLUSIONS

The comparative analysis of ESP-DL and ESP-TFLite-micro highlights that the optimal choice of an Edge AI inference engine is strictly dictated by specific application requirements and hardware constraints. ESP-DL emerges as the superior framework for systems prioritizing minimal volatile memory (SRAM) utilization. By employing a bare-metal execution strategy and a proprietary `.esddl` FlatBuffer schema, it entirely circumvents the need for a bloated interpreter engine and the monolithic Tensor Arena, resulting in an exceptionally lean footprint. Conversely, ESP-TFLite-micro offers a highly adaptable, platform-agnostic framework deeply integrated with the standard TensorFlow ecosystem. While its interpreter-based model demands a disproportionate memory allocation to prevent runtime segmentation faults, it benefits immensely from mature software-level graph optimizations, including aggressive operator fusion. Ultimately, when coupled with the ESP-NN hardware acceleration library on target silicon such as the ESP32-S3, ESP-TFLite-micro can paradoxically achieve superior inference execution speeds compared to the bare-metal ESP-DL framework, making it the preferable choice when computational speed supersedes strict SRAM limitations.